

```
In [3]: # Import Required Libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats
import openpyxl

# Display all columns
pd.set_option("display.max_columns", None)

print("All libraries imported successfully!")
```

All libraries imported successfully!

Exploratory Data Analysis (EDA)

Project 2

Student: Muhammad Afzaal

Tools Used:

- Python
- Pandas
- NumPy
- Matplotlib
- SciPy
- OpenPyXL
- Jupyter Notebook

```
In [4]: # Load Dataset

file_path = "../Data/Dataset.xlsx"

df = pd.read_excel(file_path)

print("Dataset loaded successfully!")
```

Dataset loaded successfully!

Dataset Overview

In this section, we will inspect the dataset to understand its structure, dimensions, column names, data types, and overall quality before performing any analysis.

```
In [14]: # Dataset Shape

print("Rows and Columns:")
print(df.shape)
```

Rows and Columns:
(1200, 14)

In []: *# First Five Rows*

```
df.head()
```

Out[]:

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	Pay
--	---------	------	------------	---------	----------	-----------	-----------------	-----

0	ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	
1	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	
2	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	
3	ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	
4	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	

In [15]: *# Last Five Rows*

```
df.tail()
```

Out[15]:

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress
--	---------	------	------------	---------	----------	-----------	-----------------

1195	ORD201195	2024-06-20	C21126	Desk	1	107.04	392 Main St
1196	ORD201196	2024-03-04	C20095	Monitor	2	662.53	778 Main St
1197	ORD201197	2023-07-13	C79674	Tablet	2	436.84	275 Main St
1198	ORD201198	2024-08-22	C64753	Chair	4	262.52	509 Main St
1199	ORD201199	2023-06-11	C57502	Tablet	4	560.58	201 Main St

In [16]: *# Column Names*

```
print(df.columns)
```

```
Index(['OrderID', 'Date', 'CustomerID', 'Product', 'Quantity', 'UnitPrice',  
      'ShippingAddress', 'PaymentMethod', 'OrderStatus', 'TrackingNumber',  
      'ItemsInCart', 'CouponCode', 'ReferralSource', 'TotalPrice'],  
      dtype='str')
```

In [17]: *# Dataset Information*

```
df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 1200 entries, 0 to 1199
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   OrderID               1200 non-null   str    
 1   Date                  1200 non-null   datetime64[us]
 2   CustomerID            1200 non-null   str    
 3   Product               1200 non-null   str    
 4   Quantity              1200 non-null   int64  
 5   UnitPrice             1200 non-null   float64
 6   ShippingAddress       1200 non-null   str    
 7   PaymentMethod         1200 non-null   str    
 8   OrderStatus           1200 non-null   str    
 9   TrackingNumber        1200 non-null   str    
10   ItemsInCart           1200 non-null   int64  
11   CouponCode            891 non-null    str    
12   ReferralSource        1200 non-null   str    
13   TotalPrice            1200 non-null   float64
dtypes: datetime64[us](1), float64(2), int64(2), str(9)
memory usage: 131.4 KB

```

In [18]: *# Descriptive Statistics*

```
df.describe()
```

Out[18]:

	Date	Quantity	UnitPrice	ItemsInCart	TotalPrice
count	1200	1200.000000	1200.000000	1200.000000	1200.000000
mean	2024-03-22 16:58:48	2.945833	356.412750	5.485000	1053.968300
min	2023-01-01 00:00:00	1.000000	11.390000	1.000000	11.390000
25%	2023-08-03 18:00:00	2.000000	186.062500	4.000000	410.520000
50%	2024-03-23 00:00:00	3.000000	364.210000	5.000000	823.615000
75%	2024-11-08 12:00:00	4.000000	521.570000	7.000000	1578.475000
max	2025-06-30 00:00:00	5.000000	699.930000	10.000000	3456.400000
std	NaN	1.407557	197.177146	2.281983	819.856558

In [19]: *# Missing Values*

```

missing_values = df.isnull().sum()

print(missing_values)

```

```

OrderID      0
Date         0
CustomerID   0
Product      0
Quantity     0
UnitPrice    0
ShippingAddress 0
PaymentMethod 0
OrderStatus  0
TrackingNumber 0
ItemsInCart  0
CouponCode   309
ReferralSource 0
TotalPrice   0
dtype: int64

```

In [20]: *# Duplicate Records*

```
print("Duplicate Rows:", df.duplicated().sum())
```

Duplicate Rows: 0

In []: *# Data Types*

```
print(df.dtypes)
```

```

OrderID      str
Date         datetime64[us]
CustomerID   str
Product      str
Quantity     int64
UnitPrice    float64
ShippingAddress str
PaymentMethod str
OrderStatus  str
TrackingNumber str
ItemsInCart  int64
CouponCode   str
ReferralSource str
TotalPrice   float64
dtype: object

```

Data Cleaning

In this section, we will clean the dataset by handling missing values, verifying data types, checking data consistency, and preparing the data for analysis.

In [22]: *# Data Types*

```
df.dtypes
```

```
Out[22]: OrderID          str
         Date            datetime64[us]
         CustomerID      str
         Product         str
         Quantity        int64
         UnitPrice       float64
         ShippingAddress  str
         PaymentMethod    str
         OrderStatus     str
         TrackingNumber   str
         ItemsInCart     int64
         CouponCode      str
         ReferralSource   str
         TotalPrice      float64
         dtype: object
```

```
In [ ]: # Convert Date Column

df["Date"] = pd.to_datetime(df["Date"])

print("Date column converted successfully!")
```

Date column converted successfully!

```
In [24]: # Handle Missing Coupon Codes

df["CouponCode"] = df["CouponCode"].fillna("No Coupon")

print("Missing Coupon Codes handled successfully!")
```

Missing Coupon Codes handled successfully!

```
In [25]: # Verify Missing Values

df.isnull().sum()
```

```
Out[25]: OrderID          0
         Date            0
         CustomerID      0
         Product         0
         Quantity        0
         UnitPrice       0
         ShippingAddress  0
         PaymentMethod    0
         OrderStatus     0
         TrackingNumber   0
         ItemsInCart     0
         CouponCode      0
         ReferralSource   0
         TotalPrice      0
         dtype: int64
```

```
In [26]: # Feature Engineering

df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month_name()
df["MonthNumber"] = df["Date"].dt.month
df["Quarter"] = df["Date"].dt.quarter
df["Day"] = df["Date"].dt.day_name()
```

```
print("New features created successfully!")
```

New features created successfully!

```
In [27]: df.head()
```

Out[27]:

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	Pay
0	ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	
1	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	
2	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	
3	ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	
4	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	

Analysis 1: Monthly Orders

This analysis examines the number of customer orders placed each month. It helps identify seasonal trends, peak business periods, and fluctuations in customer purchasing behavior.

Objective

- Determine the total number of orders placed each month.
- Identify months with the highest and lowest order volumes.
- Understand monthly ordering trends.

Visualization

A bar chart is used to compare the number of orders across different months.

Interpretation

The bar chart displays the monthly distribution of customer orders. Higher bars indicate months with increased customer activity, while lower bars indicate periods with fewer orders.

Key Insight

This analysis helps identify seasonal demand patterns and supports better inventory management, staffing, and marketing strategies.

In [28]: *# Monthly Orders*

```
monthly_orders = df.groupby("Month")["OrderID"].count()

monthly_orders = monthly_orders.reindex([
    "January", "February", "March", "April", "May", "June",
    "July", "August", "September", "October", "November", "December"
])

print(monthly_orders)
```

```
Month
January      106
February     106
March        128
April        113
May          120
June         147
July          87
August        79
September     73
October       78
November      76
December      87
Name: OrderID, dtype: int64
```

In [71]:

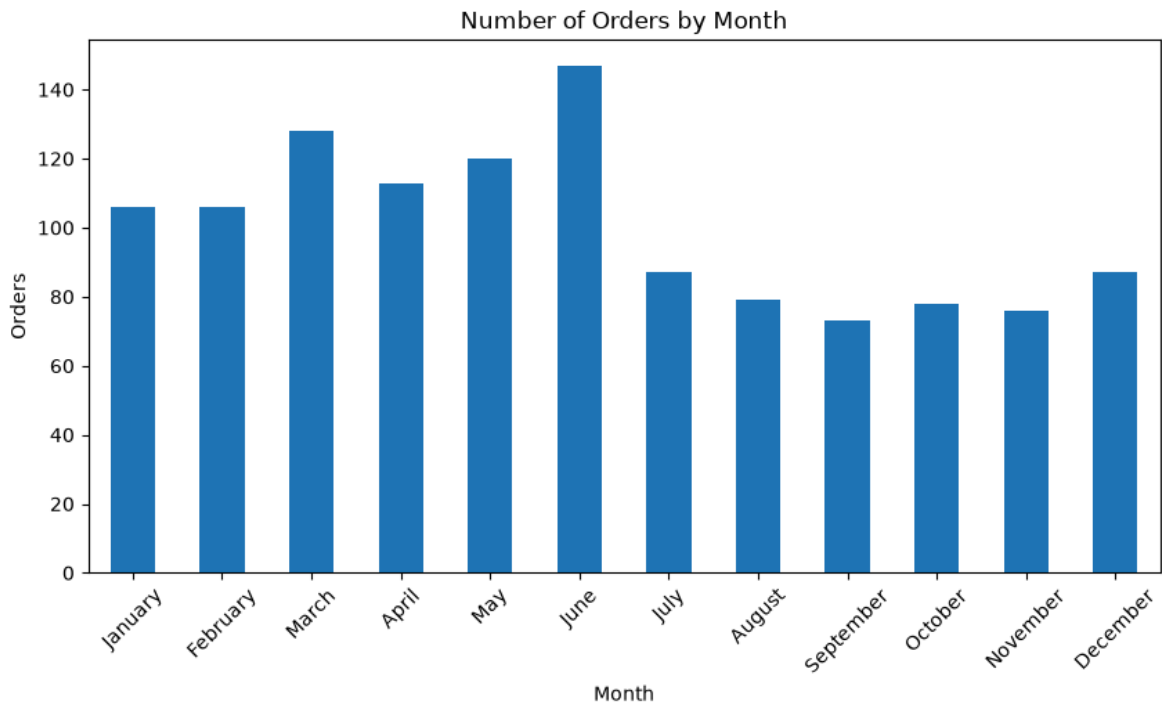
```
plt.figure(figsize=(10,5))

monthly_orders.plot(kind="bar")

plt.title("Number of Orders by Month")
plt.xlabel("Month")
plt.ylabel("Orders")
plt.xticks(rotation=45)

plt.savefig("../Charts/monthly_orders.png", dpi=300, bbox_inches="tight")

plt.show()
```



Analysis 2: Monthly Revenue

This analysis evaluates the total revenue generated each month to understand sales performance and revenue trends over time.

Objective

- Calculate total monthly revenue.
- Identify the highest and lowest revenue months.
- Analyze revenue growth or decline.

Visualization

A line chart is used to illustrate monthly revenue trends.

Interpretation

The line chart shows how revenue changes throughout the year. Peaks represent months with strong sales performance, while declines indicate weaker business periods.

Key Insight

Understanding monthly revenue trends helps businesses improve forecasting, budgeting, and sales planning.

```
In [30]: monthly_revenue = df.groupby("Month")["TotalPrice"].sum()

monthly_revenue = monthly_revenue.reindex([
    "January", "February", "March", "April", "May", "June",
    "July", "August", "September", "October", "November", "December"])
```



```
])
```

```
monthly_revenue
```

```
Out[30]: Month
January      124313.23
February     112344.78
March        123840.93
April        109186.05
May          135142.59
June         170616.13
July          85784.64
August       86343.21
September    69321.65
October      89834.82
November     75493.43
December     82540.50
Name: TotalPrice, dtype: float64
```

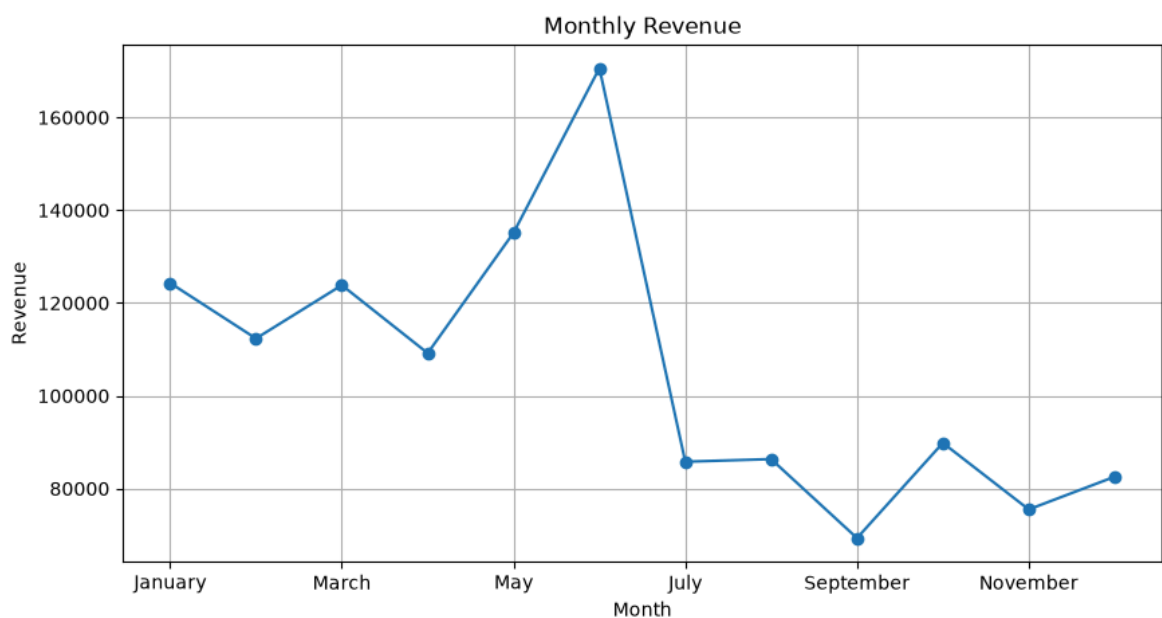
```
In [72]: plt.figure(figsize=(10,5))

monthly_revenue.plot(kind="line", marker="o")

plt.title("Monthly Revenue")
plt.xlabel("Month")
plt.ylabel("Revenue")
plt.grid(True)

plt.savefig("../Charts/monthly_revenue.png", dpi=300, bbox_inches="tight")

plt.show()
```



Analysis 3: Top 10 Best Selling Products

This analysis identifies the products with the highest sales quantity.

Objective

- Determine the best-selling products.
- Compare product demand.
- Support inventory and product planning.

Visualization

A bar chart is used to compare the top 10 products based on quantity sold.

Interpretation

The chart ranks products according to the total quantity sold. Products with the highest bars are the most popular among customers.

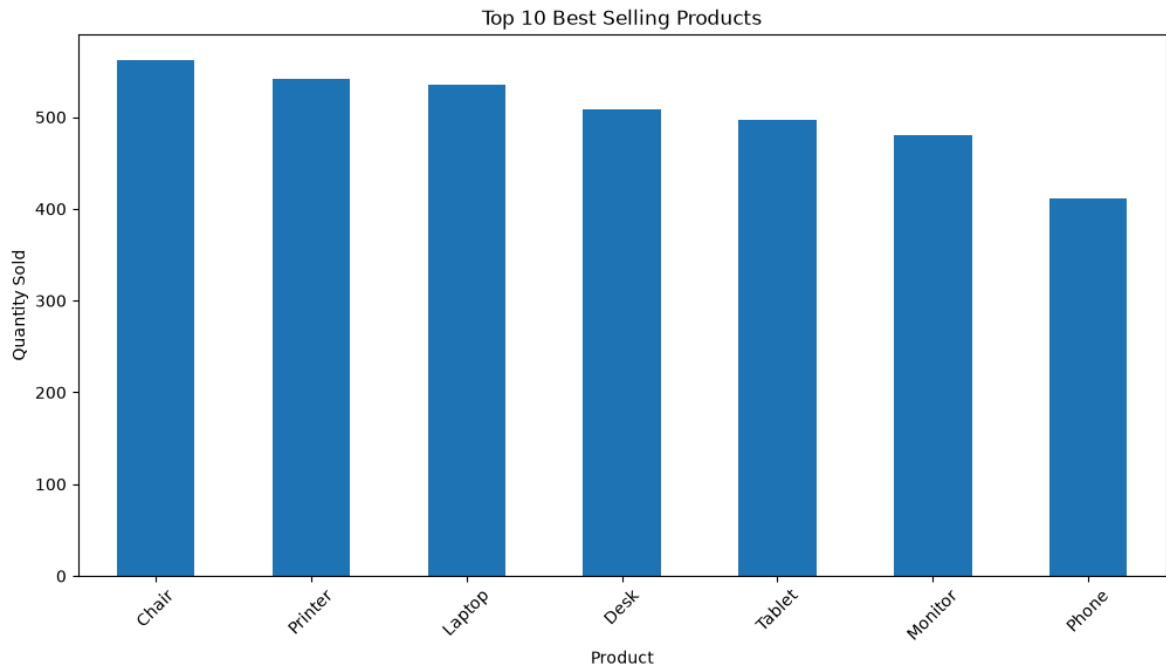
Key Insight

Best-selling products contribute significantly to overall sales and should be prioritized for inventory management and promotional activities.

```
In [32]: top_products = (  
    df.groupby("Product")["Quantity"]  
        .sum()  
        .sort_values(ascending=False)  
        .head(10)  
    )  
  
top_products
```

```
Out[32]: Product  
Chair      562  
Printer     542  
Laptop     535  
Desk       508  
Tablet     497  
Monitor    480  
Phone      411  
Name: Quantity, dtype: int64
```

```
In [73]: plt.figure(figsize=(12,6))  
  
top_products.plot(kind="bar")  
  
plt.title("Top 10 Best Selling Products")  
plt.xlabel("Product")  
plt.ylabel("Quantity Sold")  
plt.xticks(rotation=45)  
  
plt.savefig("../Charts/top_10_best_selling_products.png", dpi=300, bbox_inches="tight")  
  
plt.show()
```



Analysis 4: Payment Method Distribution

This analysis explores the payment methods preferred by customers during purchases.

Objective

- Identify the most commonly used payment methods.
- Compare customer payment preferences.
- Understand payment behavior.

Visualization

A pie chart is used to display the percentage distribution of each payment method.

Interpretation

The pie chart illustrates the share of each payment method used by customers. Larger segments indicate more popular payment options.

Key Insight

Knowing customer payment preferences helps businesses optimize payment services and improve the checkout experience.

```
In [34]: payment = df["PaymentMethod"].value_counts()  
  
payment
```

```
Out[34]: PaymentMethod
Online      258
Cash        246
Credit Card 234
Debit Card  232
Gift Card   230
Name: count, dtype: int64
```

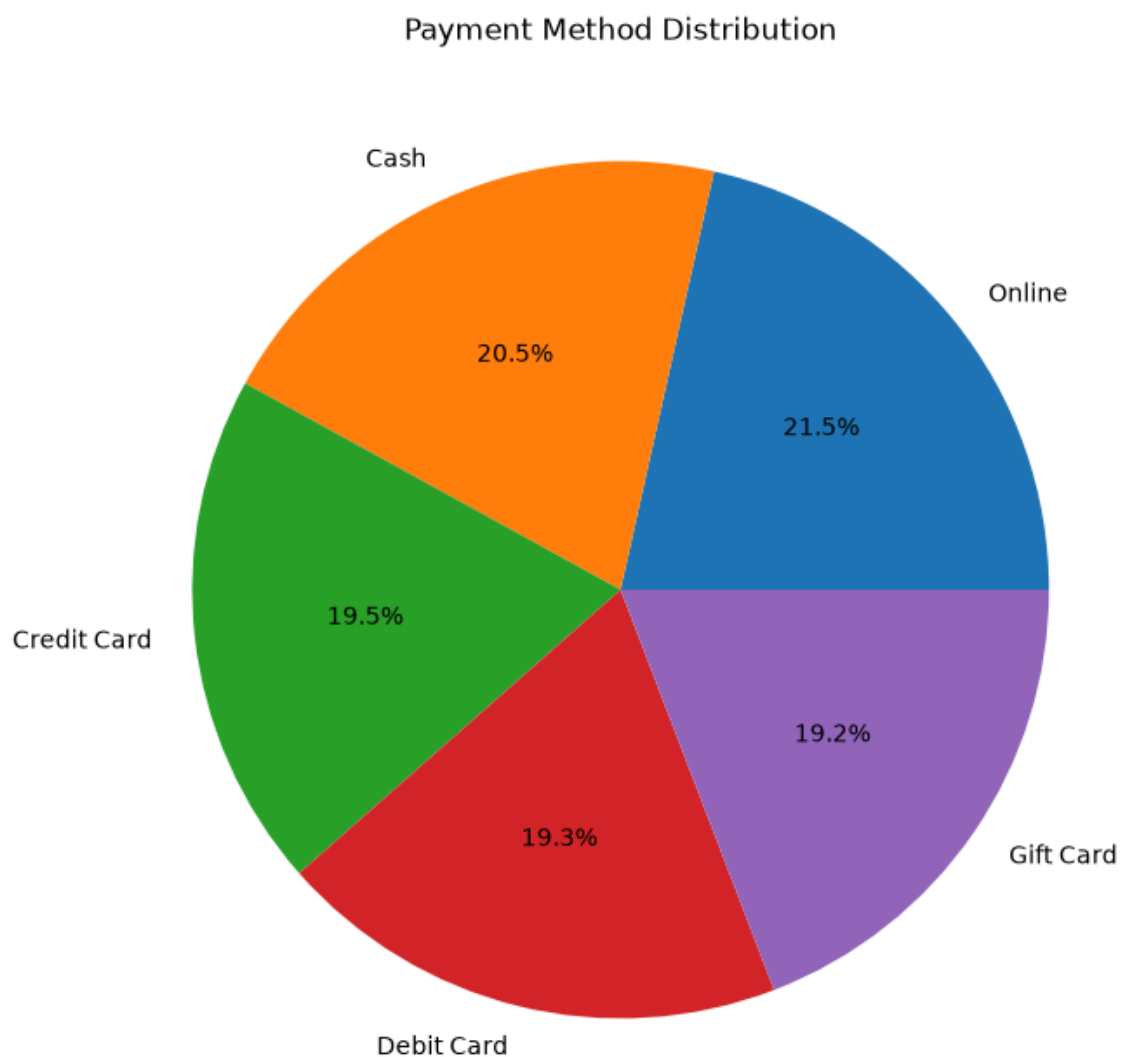
```
In [74]: plt.figure(figsize=(8,8))

payment.plot(kind="pie", autopct="%1.1f%%")

plt.ylabel("")
plt.title("Payment Method Distribution")

plt.savefig("../Charts/payment_method_distribution.png", dpi=300, bbox_inches="tight")

plt.show()
```



Analysis 5: Order Status Distribution

This analysis examines the distribution of different order statuses.

Objective

- Determine the number of delivered, pending, and cancelled orders.
- Evaluate order fulfillment performance.
- Identify operational trends.

Visualization

A bar chart is used to compare the frequency of each order status.

Interpretation

The chart shows the number of orders in each status category. A higher number of delivered orders indicates successful order processing.

Key Insight

Monitoring order status helps businesses improve operational efficiency and customer satisfaction.

```
In [36]: status = df["OrderStatus"].value_counts()

status
```

```
Out[36]: OrderStatus
Cancelled      250
Returned       247
Pending        237
Shipped        235
Delivered      231
Name: count, dtype: int64
```

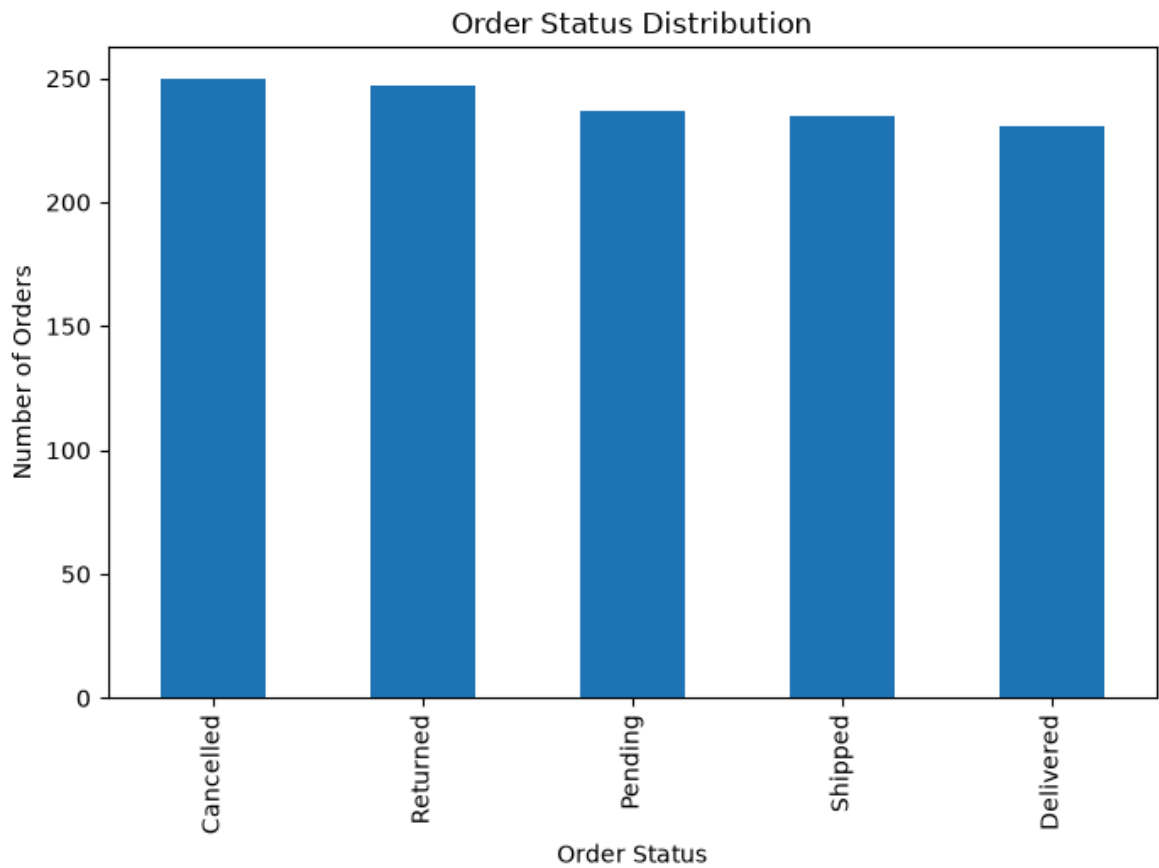
```
In [75]: plt.figure(figsize=(8,5))

status.plot(kind="bar")

plt.title("Order Status Distribution")
plt.xlabel("Order Status")
plt.ylabel("Number of Orders")

plt.savefig("../Charts/order_status_distribution.png", dpi=300, bbox_inches="tight")

plt.show()
```



Analysis 6: Referral Source Analysis

This analysis investigates the sources through which customers reached the online store.

Objective

- Identify the most effective referral channels.
- Compare customer acquisition sources.
- Evaluate marketing performance.

Visualization

A bar chart is used to compare referral sources.

Interpretation

The chart highlights which referral sources contribute the highest number of customers.

Key Insight

The most successful referral channels should receive greater marketing investment to maximize customer acquisition.

```
In [38]: referral = df["ReferralSource"].value_counts()
```

```
referral
```

```
Out[38]: ReferralSource
Instagram    259
Email        250
Google       241
Facebook     228
Referral     222
Name: count, dtype: int64
```

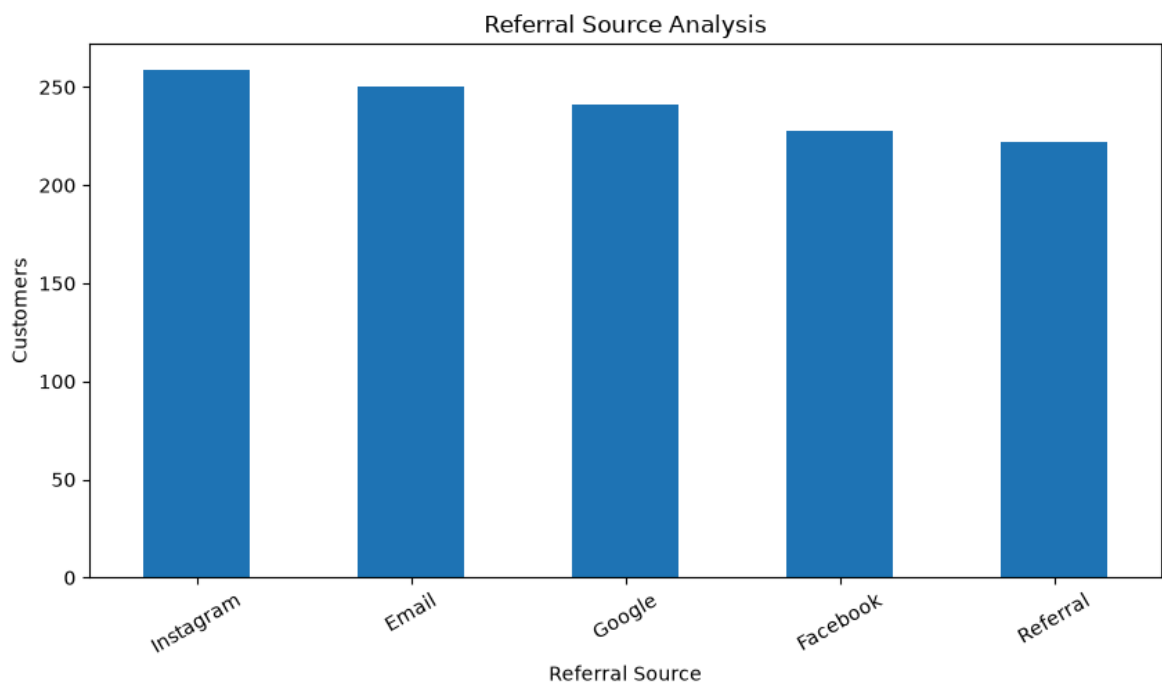
```
In [76]: plt.figure(figsize=(10,5))

referral.plot(kind="bar")

plt.title("Referral Source Analysis")
plt.xlabel("Referral Source")
plt.ylabel("Customers")
plt.xticks(rotation=30)

plt.savefig("../Charts/referral_source_analysis.png", dpi=300, bbox_inches="tight")

plt.show()
```



Analysis 7: Coupon Usage Analysis

This analysis evaluates the usage of promotional coupon codes by customers.

Objective

- Identify the most frequently used coupon codes.
- Compare coupon usage.
- Measure customer response to promotions.

Visualization

A bar chart is used to display the most frequently used coupon codes.

Interpretation

The chart presents the popularity of different coupon codes. Frequently used coupons indicate successful promotional campaigns.

Key Insight

Coupon analysis helps businesses evaluate the effectiveness of marketing promotions and customer engagement strategies.

```
In [40]: coupon = df["CouponCode"].value_counts()

coupon.head(10)
```

```
Out[40]: CouponCode
FREESHIP      313
No Coupon     309
WINTER15      292
SAVE10        286
Name: count, dtype: int64
```

```
In [77]: plt.figure(figsize=(10,5))

coupon.head(10).plot(kind="bar")

plt.title("Top Coupon Codes")
plt.xlabel("Coupon Code")
plt.ylabel("Usage")
plt.xticks(rotation=45)

plt.savefig("../Charts/coupon_usage_analysis.png", dpi=300, bbox_inches="tight")

plt.show()
```




Analysis 8: Quantity Distribution

This analysis examines the distribution of product quantities ordered by customers.

Objective

- Understand purchasing patterns.
- Identify the most common order quantities.
- Detect unusual purchasing behavior.

Visualization

A histogram is used to display the frequency distribution of ordered quantities.

Interpretation

The histogram shows how frequently different quantities occur in customer orders.

Key Insight

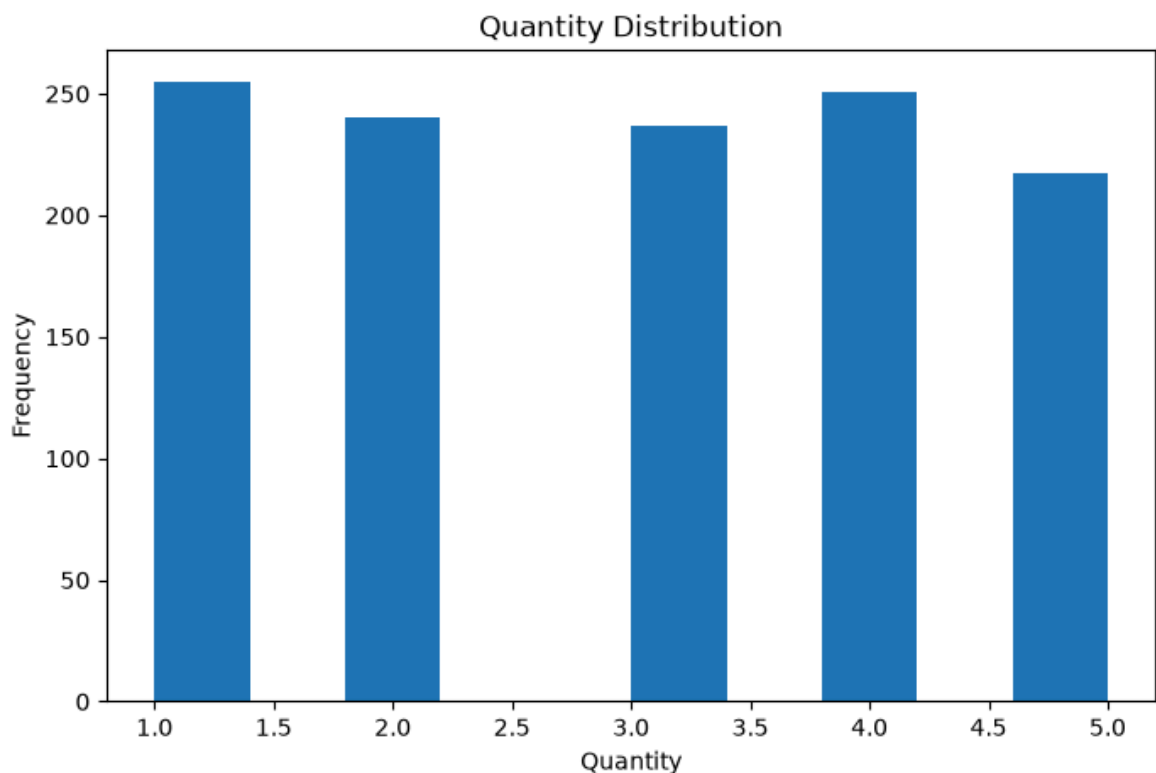
Understanding quantity distribution supports inventory planning and demand forecasting.

```
In [78]: plt.figure(figsize=(8,5))

plt.hist(df["Quantity"], bins=10)

plt.title("Quantity Distribution")
plt.xlabel("Quantity")
plt.ylabel("Frequency")
```

```
plt.savefig("../Charts/quantity_distribution.png", dpi=300, bbox_inches="tight")  
  
plt.show()
```



Analysis 9: Total Price Distribution

This analysis explores the distribution of total order values.

Objective

- Understand customer spending patterns.
- Identify common purchase values.
- Detect unusually high-value orders.

Visualization

A histogram is used to display the distribution of total order prices.

Interpretation

The histogram illustrates how customer spending is distributed across all orders.

Key Insight

Analyzing spending patterns helps businesses understand customer purchasing behavior and pricing strategies.

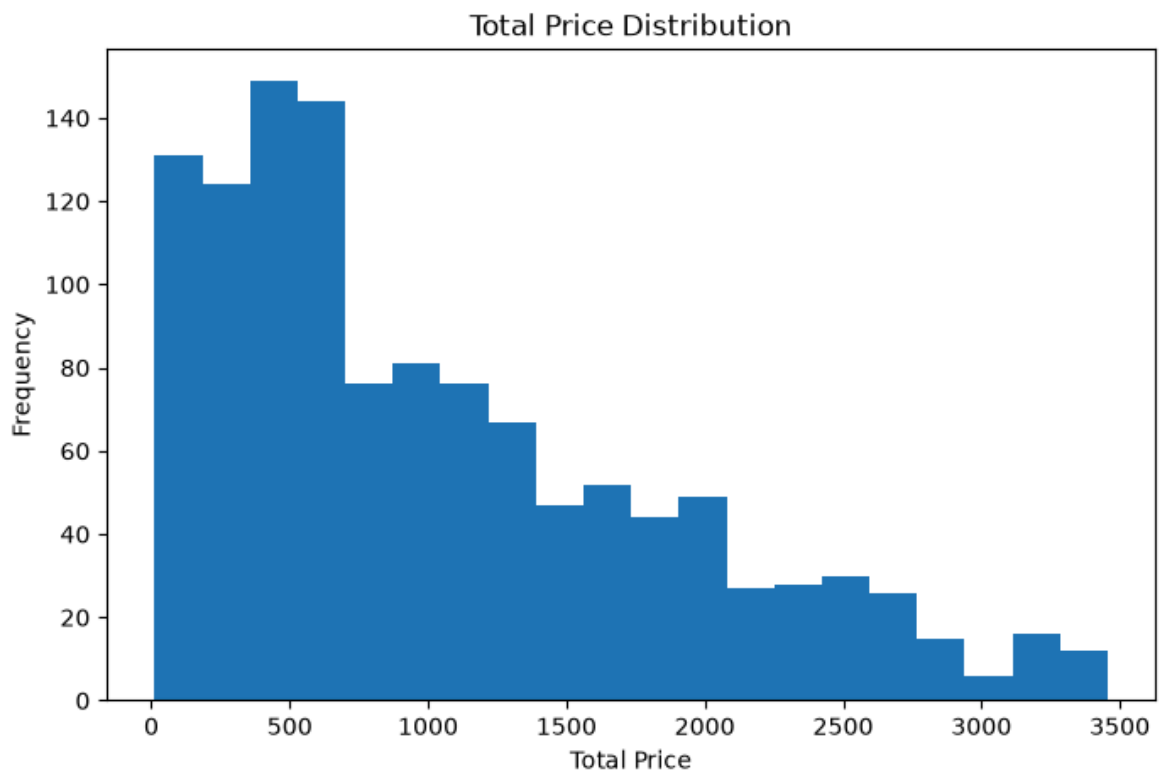
```
In [79]: plt.figure(figsize=(8,5))

plt.hist(df["TotalPrice"], bins=20)

plt.title("Total Price Distribution")
plt.xlabel("Total Price")
plt.ylabel("Frequency")

plt.savefig("../Charts/total_price_distribution.png", dpi=300, bbox_inches="tight")

plt.show()
```



Analysis 10: Unit Price vs Total Price

This analysis investigates the relationship between unit price and total order price.

Objective

- Examine the relationship between product price and total purchase value.
- Identify trends in customer spending.
- Detect unusual observations.

Visualization

A scatter plot is used to visualize the relationship between unit price and total price.

Interpretation

Each point represents an order. A positive relationship indicates that higher-priced products generally contribute to higher total order values.

Key Insight

Understanding this relationship assists businesses in pricing decisions and sales performance analysis.

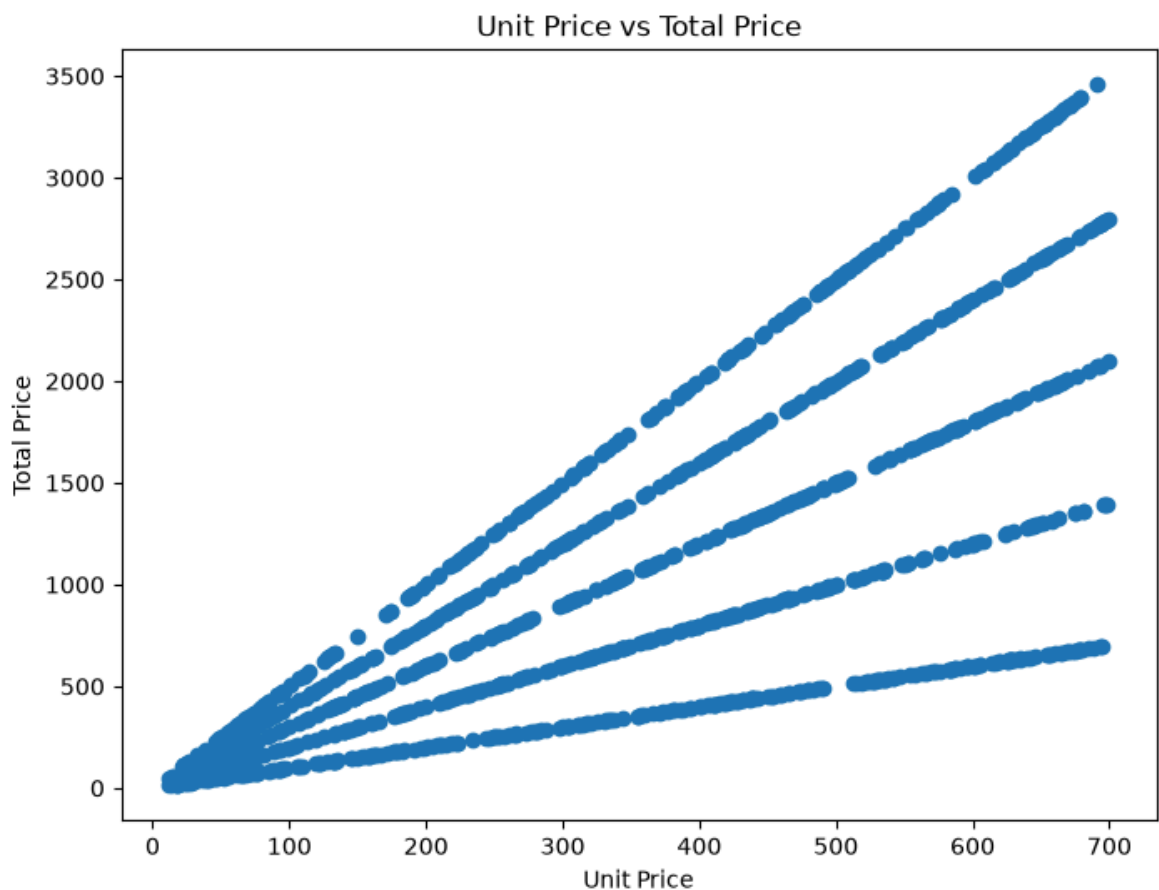
```
In [80]: plt.figure(figsize=(8,6))

plt.scatter(df["UnitPrice"], df["TotalPrice"])

plt.title("Unit Price vs Total Price")
plt.xlabel("Unit Price")
plt.ylabel("Total Price")

plt.savefig("../Charts/unitprice_vs_totalprice.png", dpi=300, bbox_inches="tight")

plt.show()
```



Analysis 11: Correlation Analysis

This analysis measures the relationship between numerical variables in the dataset.

Objective

- Identify relationships among numerical variables.
- Detect positive and negative correlations.

- Support data-driven decision making.

Visualization

A correlation heatmap is used to visualize relationships between numerical features.

Interpretation

The heatmap displays correlation coefficients ranging from -1 to +1. Values closer to +1 indicate strong positive relationships, while values closer to -1 indicate strong negative relationships.

Key Insight

Correlation analysis helps identify factors that influence sales performance and customer purchasing behavior.

```
In [45]: numeric_df = df.select_dtypes(include=np.number)

numeric_df.corr()
```

```
Out[45]:
```

	Quantity	UnitPrice	ItemsInCart	TotalPrice	Year	MonthNumber
Quantity	1.000000	0.014553	0.650061	0.615251	-0.033229	-0.015118
UnitPrice	0.014553	1.000000	0.000602	0.717081	-0.002233	-0.026130
ItemsInCart	0.650061	0.000602	1.000000	0.392540	-0.050464	-0.018151
TotalPrice	0.615251	0.717081	0.392540	1.000000	-0.036174	-0.027734
Year	-0.033229	-0.002233	-0.050464	-0.036174	1.000000	-0.269465
MonthNumber	-0.015118	-0.026130	-0.018151	-0.027734	-0.269465	1.000000
Quarter	-0.013799	-0.026416	-0.018386	-0.023235	-0.295158	0.969097

```
In [81]: plt.figure(figsize=(10,8))

plt.imshow(numeric_df.corr(), cmap="coolwarm")

plt.colorbar()

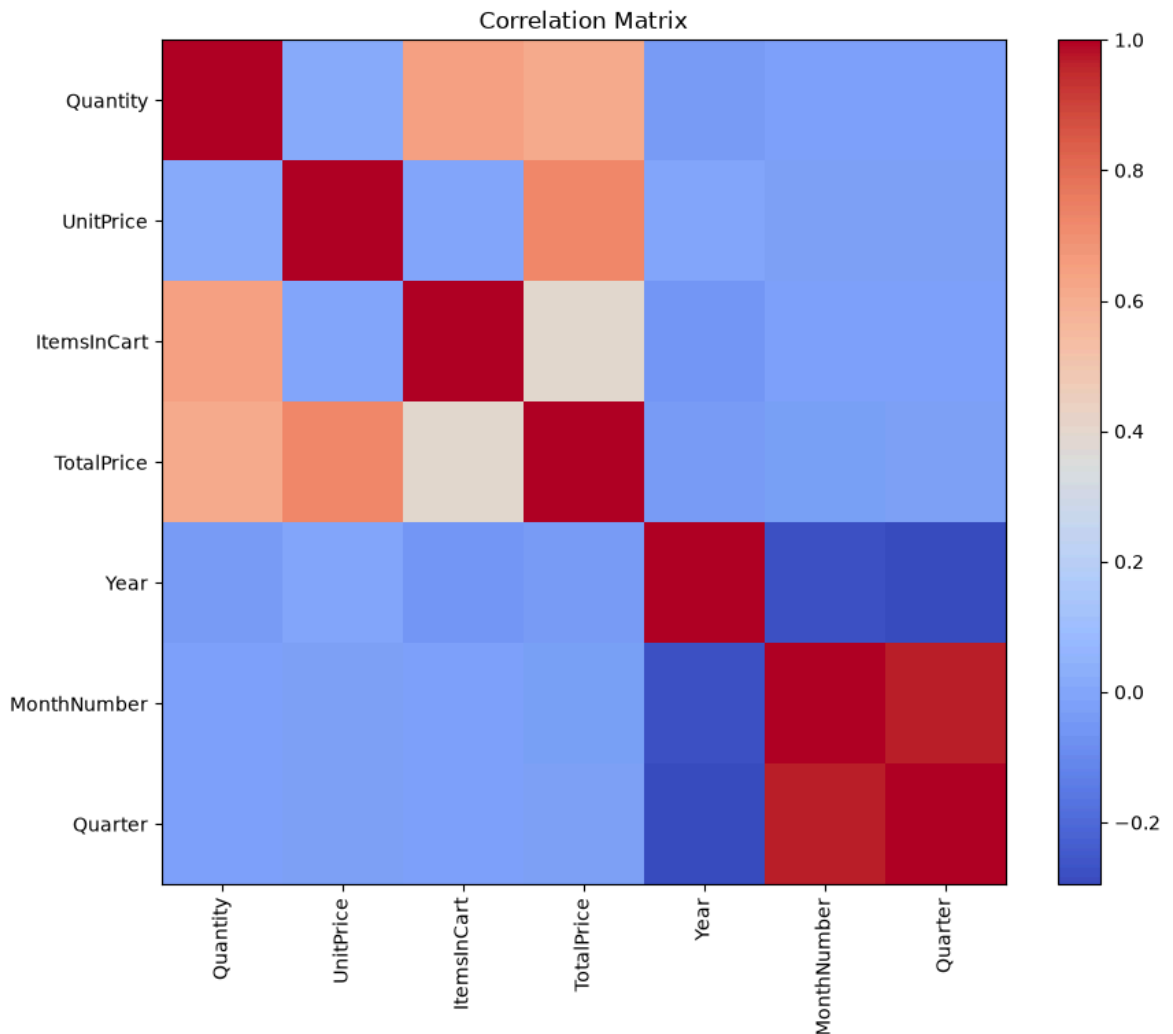
plt.xticks(
    range(len(numeric_df.columns)),
    numeric_df.columns,
    rotation=90
)

plt.yticks(
    range(len(numeric_df.columns)),
    numeric_df.columns
)
```

```
plt.title("Correlation Matrix")

plt.savefig("../Charts/correlation_heatmap.png", dpi=300, bbox_inches="tight")

plt.show()
```



Analysis 12: Outlier Detection

This analysis identifies unusually high or low values within the numerical data.

Objective

- Detect outliers in Quantity and Total Price.
- Identify abnormal customer purchasing behavior.
- Improve data quality for analysis.

Visualization

Boxplots are used to identify outliers in numerical variables.

Interpretation

Values outside the whiskers represent potential outliers. These observations may require further investigation.

Key Insight

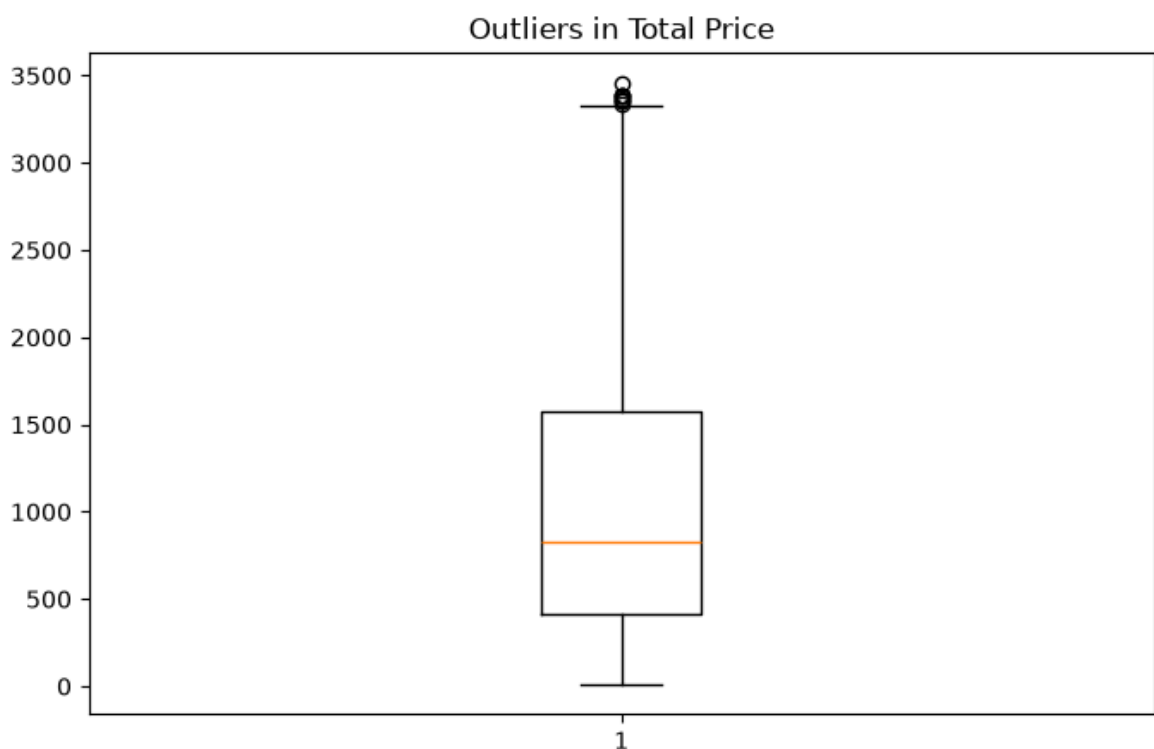
Outlier detection improves the reliability of statistical analysis and helps identify exceptional business events or data entry issues.

```
In [82]: plt.figure(figsize=(8,5))

plt.boxplot(df["TotalPrice"])

plt.title("Outliers in Total Price")
plt.savefig("../Charts/outliers_totalprice.png", dpi=300, bbox_inches="tight")

plt.show()
```



Statistical Analysis

This section performs statistical analysis on the numerical variables to better understand the central tendency, variability, and relationships within the dataset.

```
In [49]: # Summary Statistics

numerical_columns = ["Quantity", "UnitPrice", "TotalPrice"]

df[numerical_columns].describe().T
```

Out[49]:

	count	mean	std	min	25%	50%	75%	max
Quantity	1200.0	2.945833	1.407557	1.00	2.0000	3.000	4.000	5.00
UnitPrice	1200.0	356.412750	197.177146	11.39	186.0625	364.210	521.570	699.93
TotalPrice	1200.0	1053.968300	819.856558	11.39	410.5200	823.615	1578.475	3456.40

In [50]:

```
# Skewness

print("Skewness:\n")
print(df[numerical_columns].skew())
```

Skewness:

```
Quantity      0.027922
UnitPrice    -0.026512
TotalPrice     0.891359
dtype: float64
```

In [51]:

```
# Kurtosis

print("Kurtosis:\n")
print(df[numerical_columns].kurt())
```

Kurtosis:

```
Quantity      -1.294596
UnitPrice     -1.191011
TotalPrice    -0.040415
dtype: float64
```

In [52]:

```
# Correlation Matrix

corr = df[numerical_columns].corr()

corr
```

Out[52]:

	Quantity	UnitPrice	TotalPrice
Quantity	1.000000	0.014553	0.615251
UnitPrice	0.014553	1.000000	0.717081
TotalPrice	0.615251	0.717081	1.000000

Business Insights

Based on the exploratory data analysis, the following business insights were identified.

Key Business Insights

- Monthly sales fluctuate throughout the year, indicating seasonal purchasing behavior.

- A small number of products contribute significantly to total sales.
- Most customers prefer one or two payment methods over others.
- The majority of orders are successfully delivered, indicating efficient order fulfillment.
- Certain referral sources generate more customers, highlighting effective marketing channels.
- Many customers complete purchases without using coupon codes.
- Total order value generally increases with higher product prices and larger purchase quantities.
- A few outliers represent unusually large orders, which may correspond to bulk purchases or premium customers.

Conclusion

The exploratory data analysis provided valuable insights into customer purchasing behavior, sales trends, payment preferences, and product performance.

The dataset was successfully cleaned, transformed, and analyzed using Python and various data analysis libraries. Visualizations revealed important trends that can support business decision-making.

Overall, the analysis demonstrates how data-driven approaches can improve operational efficiency, customer satisfaction, and strategic planning.

Recommendations

Based on the findings, the following recommendations are suggested:

1. Increase inventory for best-selling products.
2. Launch promotional campaigns during low-sales months.
3. Focus marketing efforts on the highest-performing referral channels.
4. Improve customer engagement through personalized coupon campaigns.
5. Monitor cancelled orders to reduce order failure rates.
6. Analyze high-value customers separately for loyalty programs.
7. Continue monitoring sales trends for better demand forecasting.
8. Regularly perform exploratory data analysis to support business decision-making.

Export Clean Dataset

The cleaned dataset is saved for future analysis and machine learning tasks.

```
In [53]: # Save Clean Dataset
```

```
df.to_excel("../Outputs/Cleaned_Dataset.xlsx", index=False)  
print("Cleaned dataset saved successfully!")
```

Cleaned dataset saved successfully!

```
In [70]: corr.to_excel("../Outputs/Correlation_Matrix.xlsx")
```

```
print("Correlation matrix saved!")
```

Correlation matrix saved!

```
In [ ]:
```